

Natural Language Processing and Large Language Models

Xiaojun Quan and Zhengjun Yue

Shenzhen Loop Area Institute

Autumn, 2026

Opening

AI is everywhere now. It writes our emails, recommends our movies, and saves us hours on boring tasks. I use it every day, and it's amazing.

But here's the thing: AI can give answers, but it doesn't know which questions matter. It can mimic empathy, but it doesn't feel it.

So use it, learn from it—but don't let it make your decisions for you. Because being human isn't about being efficient. It's about experiencing life, making mistakes, and caring about the people and things that matter to us. And no algorithm can do that.

Let AI be our tool, not our master.

Opening

AI is everywhere now. It writes our emails, recommends our movies, and saves us hours on boring tasks. I use it every day, and it's amazing.

But here's the thing: AI can give answers, but it doesn't know which questions matter. It can mimic empathy, but it doesn't feel it.

So use it, learn from it—but don't let it make your decisions for you. Because being human isn't about being efficient. It's about experiencing life, making mistakes, and caring about the people and things that matter to us. And no algorithm can do that.

Let AI be our tool, not our master.

Recap

- ❖ AI is **reshaping** life and work.
- ❖ NLP now extends to **code** and **multimodal** sequences.
- ❖ **Language model** assigns probabilities to token sequences.
- ❖ LLMs learn patterns during **training** and generate token by token during **inference**.
- ❖ **Context** shapes the probability distribution over possible continuations.

Learning Objectives

Explain the path from text to tokens, token IDs, and embeddings.

1. Compare character-, word-, subword-, and byte-level tokenization.
2. Explain why tokenizer design affects efficiency, coverage, and model behavior.
3. Distinguish token embeddings from contextual representations.

Outline

- ❑ From Text To Computer Representation
- ❑ From Unlimited Text To A Finite Vocabulary
- ❑ Building Tokenizers And Vocabularies
- ❑ From Token Ids To Embeddings
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ Takeaways

Outline

- ❑ **FROM TEXT TO COMPUTER REPRESENTATION**
- ❑ From Unlimited Text To A Finite Vocabulary
- ❑ Building Tokenizers And Vocabularies
- ❑ From Token Ids To Embeddings
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ Takeaways

How do computers process the following paragraph?

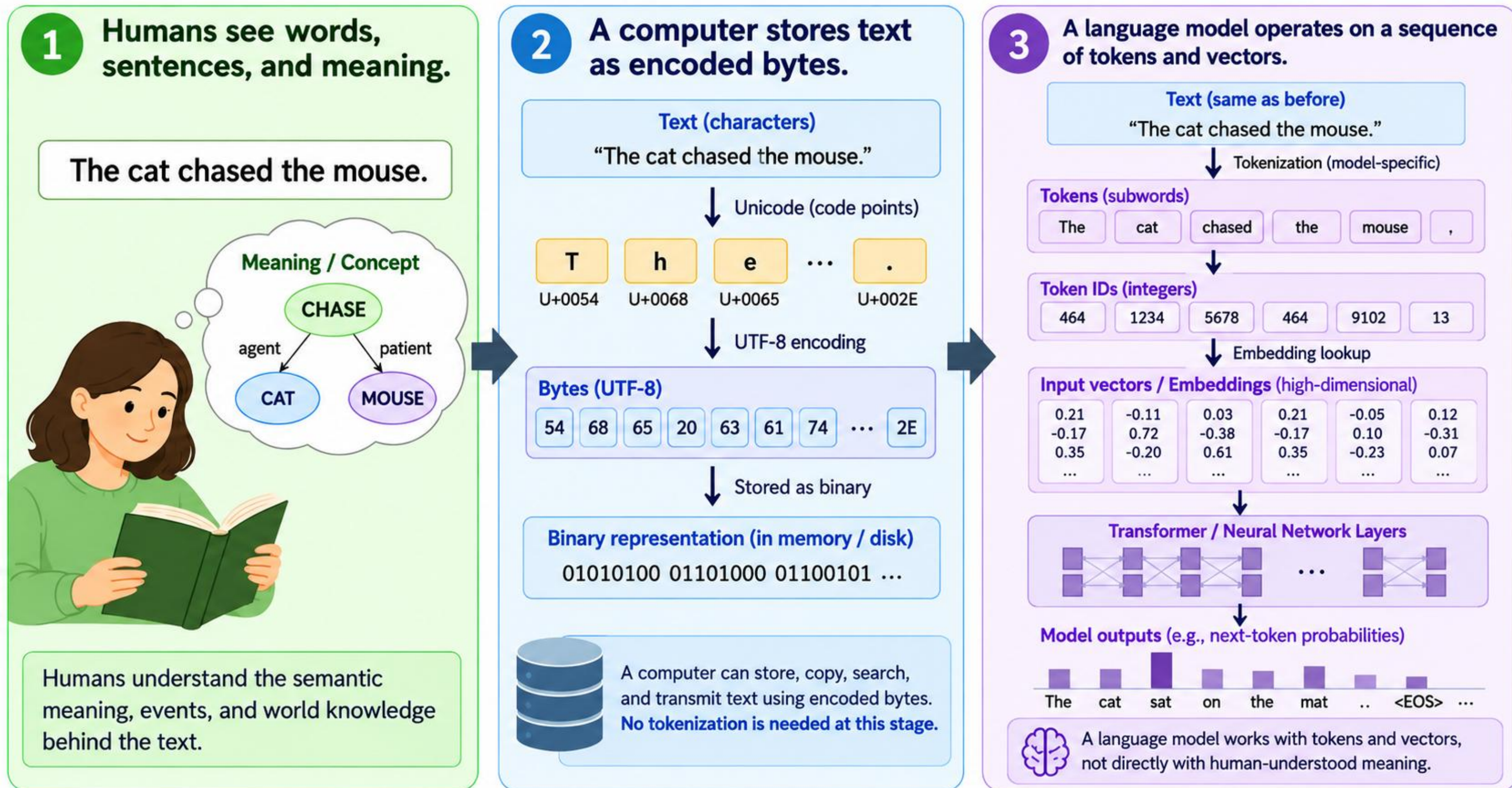
“AI is everywhere now. It writes our emails, recommends our movies, and saves us hours on boring tasks. I use it every day, and it's amazing. But here's the thing: AI can give answers, but it doesn't know which questions matter. It can mimic empathy, but it doesn't feel it. So use it, learn from it—but don't let it make your decisions for you. Because being human isn't about being efficient. It's about experiencing life, making mistakes, and caring about the people and things that matter to us. And no algorithm can do that. Let AI be our tool, not our master.”

Motivation: Computer Encoding

Computers don't understand language the way humans do.

- ❑ Modern software allows the end user to **ignore** this, but at the *lowest levels of your computer*, everything is represented by a *binary* (二进制) electrical signal that registers in one of two states: **on** or **off**.
- ❑ **Computers process and store everything as binary digits (1/0).**
 - **Bit** (比特): Each **1** or **0** is called a “bit”, from binary digit.
 - **Byte** (字节): A group of 8 bits together is called a “**byte**”, e.g., 1100 1000.
 - Example, “**a**” could be represented as **1010 0001** in computers.

Human vs Computer vs Model

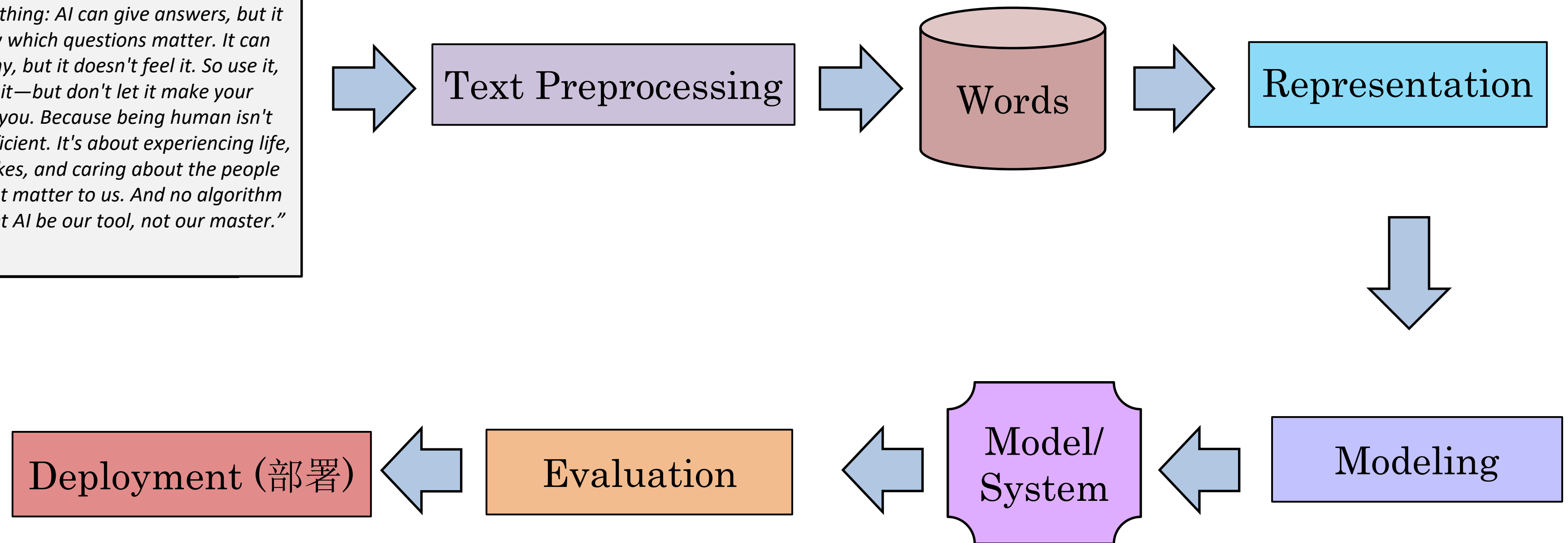


Encoding tells a computer how to store text; tokenization tells a language model what units to process.

How do computers process the following paragraph?

"AI is everywhere now. It writes our emails, recommends our movies, and saves us hours on boring tasks. I use it every day, and it's amazing. But here's the thing: AI can give answers, but it doesn't know which questions matter. It can mimic empathy, but it doesn't feel it. So use it, learn from it—but don't let it make your decisions for you. Because being human isn't about being efficient. It's about experiencing life, making mistakes, and caring about the people and things that matter to us. And no algorithm can do that. Let AI be our tool, not our master."

Pipeline of Natural Language Processing

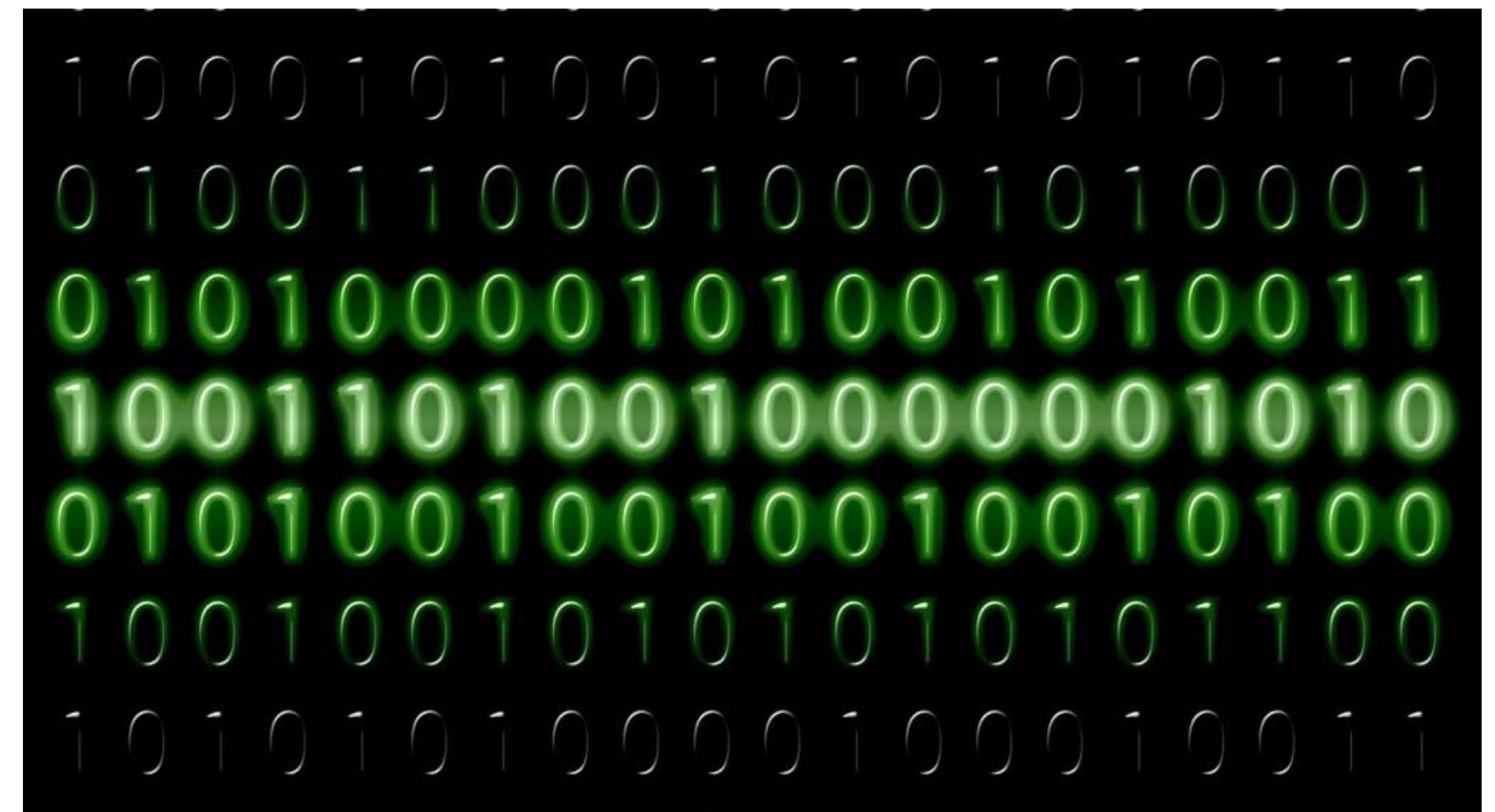


Outline

- ❑ From Text to Computer Representation
- ❑ **FROM UNLIMITED TEXT TO A FINITE VOCABULARY**
- ❑ Building Tokenizers and Vocabularies
- ❑ From Token IDs to Embeddings
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ Takeaways

Text Preprocessing: Motivation

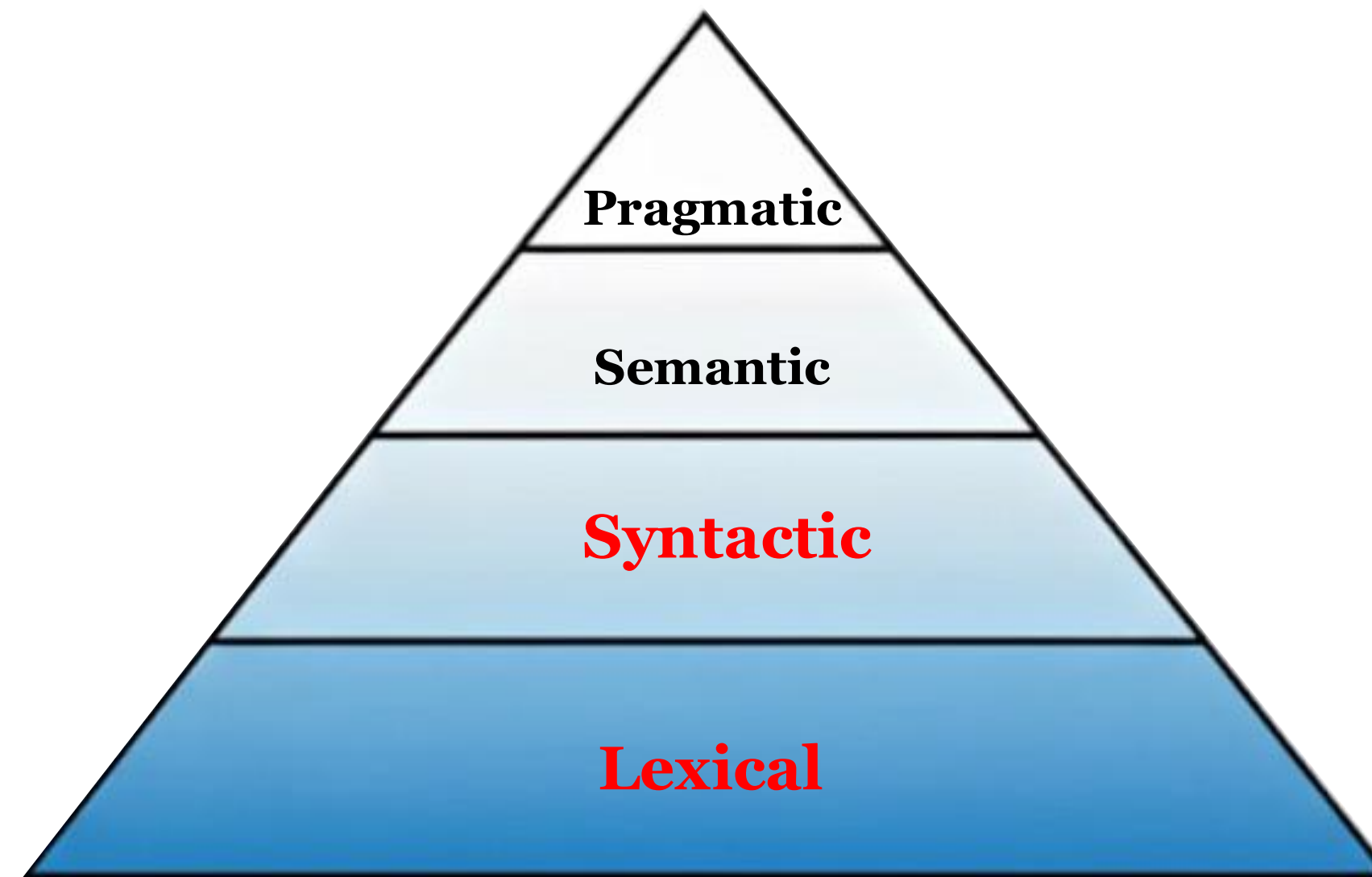
- ❑ **Human Language Understanding** is an active, dynamic process of **meaning construction** based on *background knowledge*.
- ❑ **Computer's view**: a long string or a stream of bits. **Semantics is completely absent at this level**. This is the “Semantic Gap” (语义鸿沟).



(Source: Google Image)

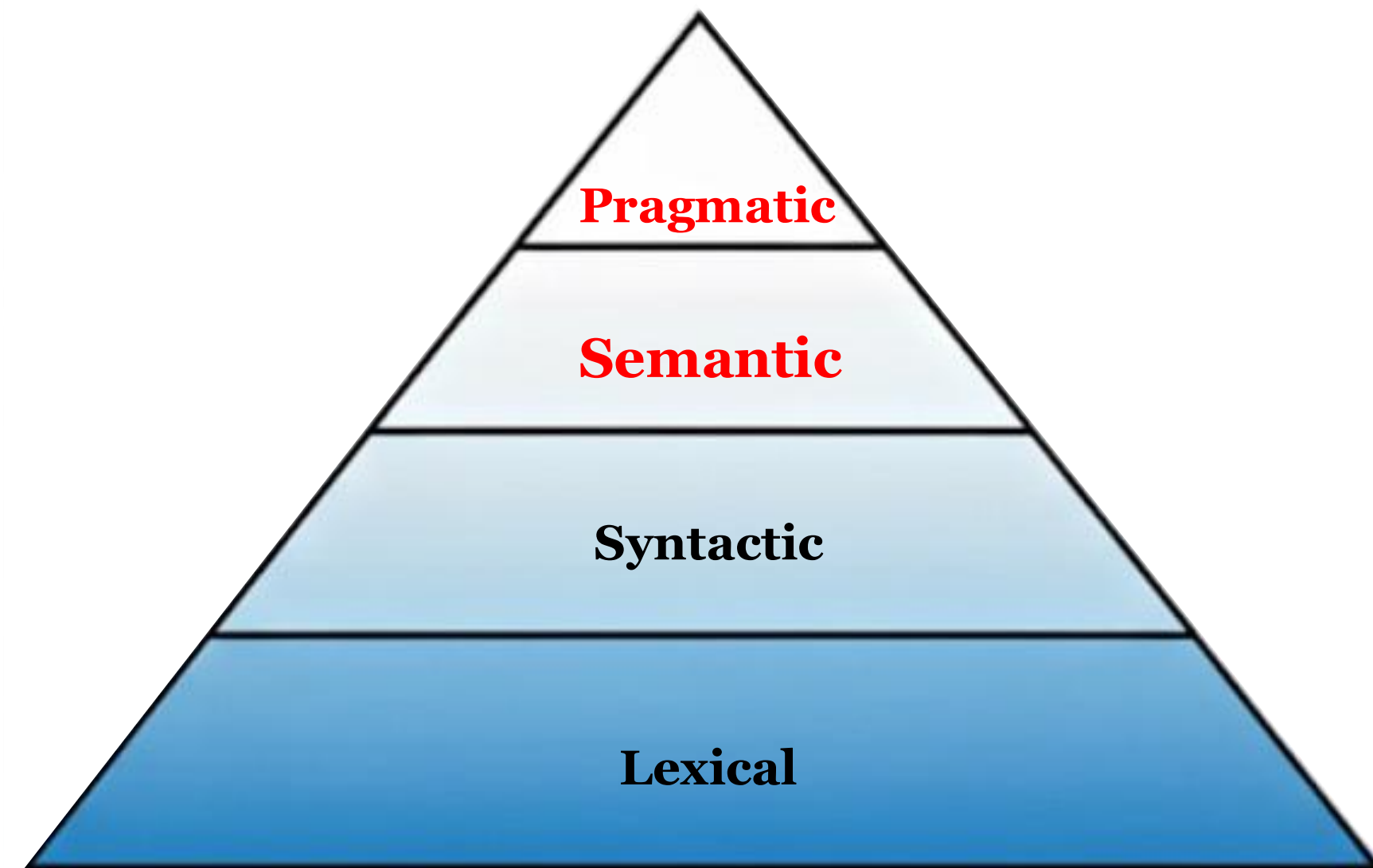
Language Understanding: **Human**

- The **hierarchical processing mechanism** of human understanding
- 1. Lexical** (词汇层): Recognize **words** and retrieving their basic **meanings**.
 - 2. Syntactic** (句法层): Analyze **sentence structure** to understand grammatical relationships between words.



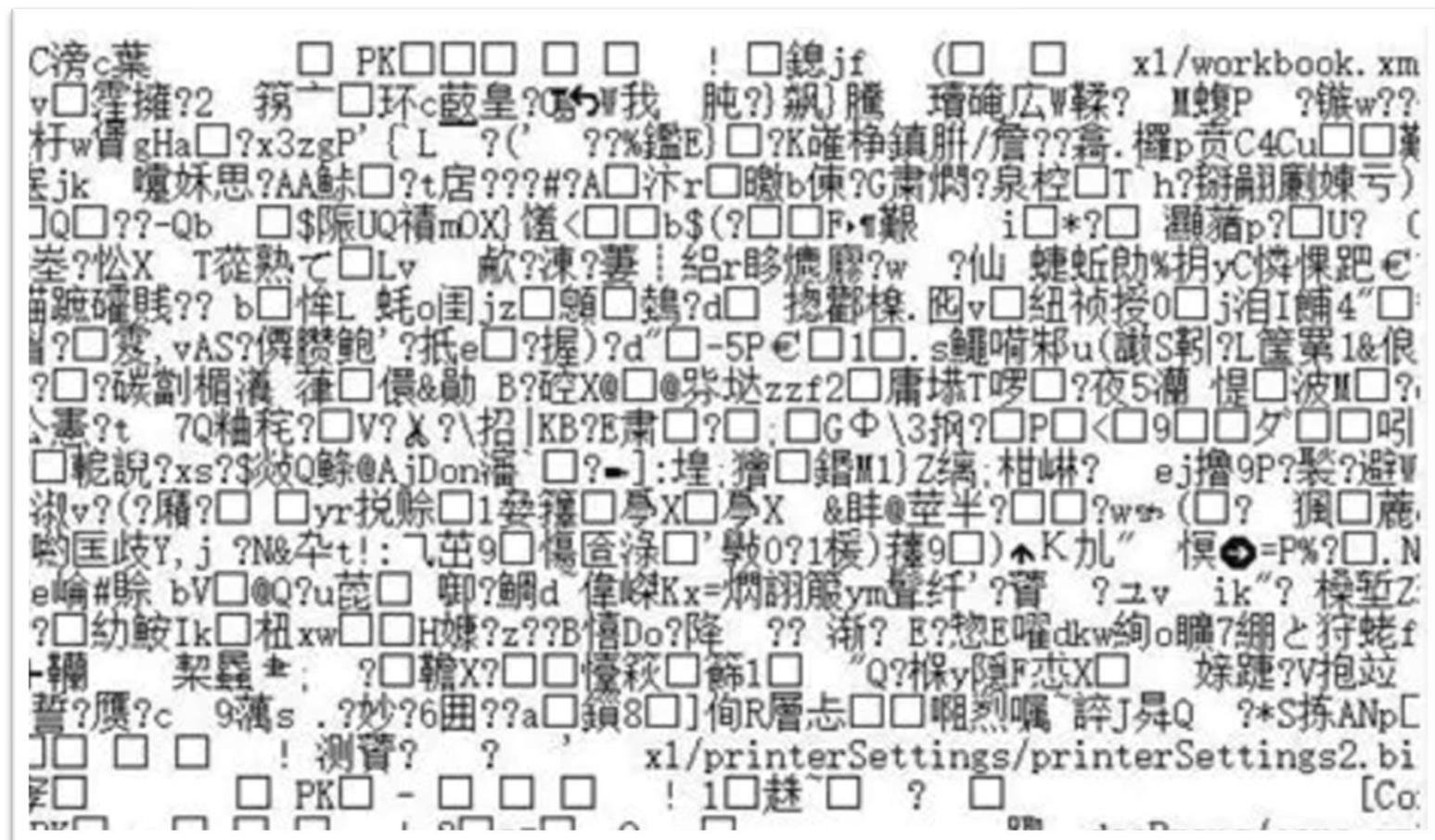
Language Understanding: **Human**

- ❑ The **hierarchical** processing mechanism of human understanding.
 - 3. Semantic** (语义层): Integrate lexical meaning and syntax to form the *literal meaning* (字面意思) of a sentence.
 - 4. Pragmatic** (语用层): Combine context to infer the speaker's *true intention*.



Text Preprocessing: Motivation

- ❑ **Messy Data:** inconsistent **formats** (case, punctuation, HTML tags)
 - Example: *<p>Click here for a discount!</p>*
- ❑ **Harmful or Biased Data:** Text containing **racial slurs** (种族诽谤) or other **toxic** (有毒) content.



Text Preprocessing: Motivation

❑ Redundant Data: Meaningless repetitions (重复) characters

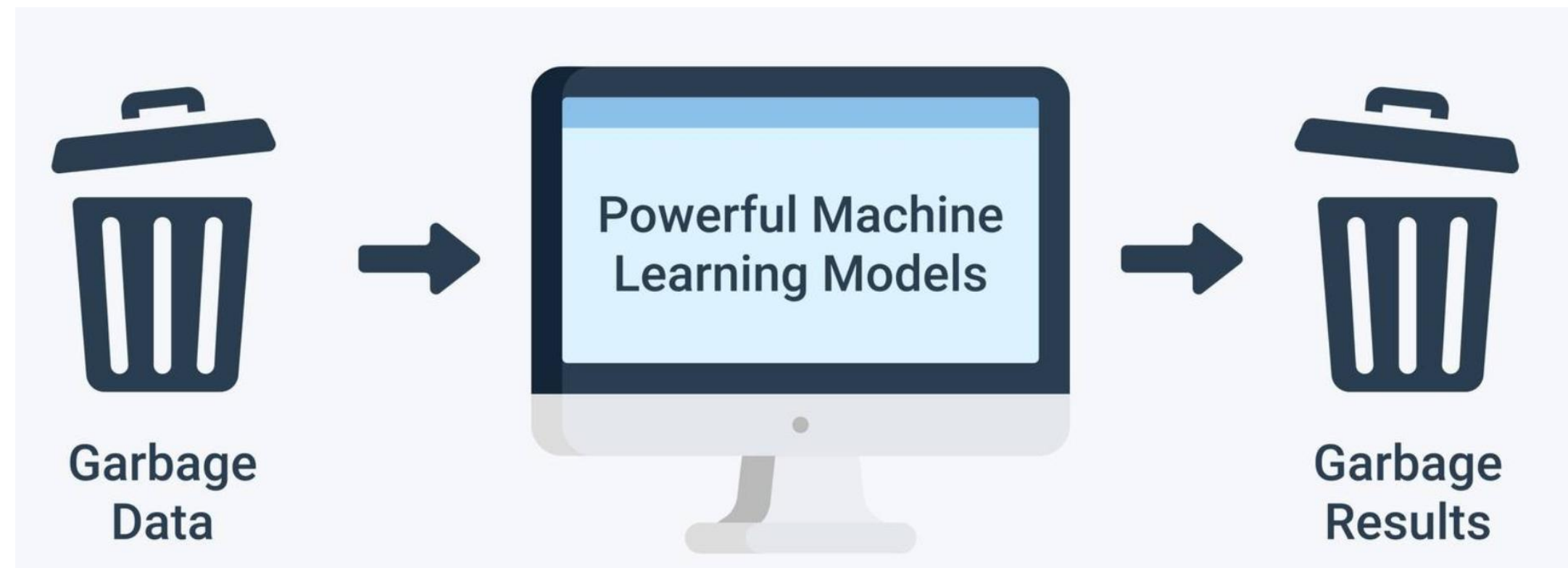
- Example: *error error error 123!@#\$*
- Example: *End of report. End of report. End of report.*

❑ Grammatical & Spelling Errors: Language mistakes.

- Spelling Error: *goot good study and dey day up.*
- Grammatical Error : *He go to school every day.*

Text Preprocessing: **Motivation**

- ❑ **“Garbage In, Garbage Out”**: The quality of the input data directly determines the **quality of the model's output**.
 - *Example*: Models trained on *biased data* will generate *biased outputs*.



Text Preprocessing: **Motivation**

Training Efficiency: A smaller, more manageable **vocabulary and training data** can speed up model training and reduces memory usage.



Text Preprocessing: Noise Removal & Cleaning

- ❑ **Removing Noise:** Eliminating **HTML** tags, **URLs**, and **stopwords** (common words like “and”, “the”, “is”).
 - *Example:* “Go” → “Go”.
- ❑ **Spelling Correction:** Fixes **typos**, which reduces the number of out-of-vocabulary (OOV) words.
 - *Example:* “I recieved the package.” → “I received the package.”
- ❑ **Handling Repeated Characters:** Normalizes elongated words common in informal text.
 - *Example:* “This is sooooo gooddd!” → “This is so good!”

Text Preprocessing: Domain-Specific Text

■ Code:

- Code block preservation
- Inline code formatting

■ Academic:

- formula & equation handling

cpp

```
1 // 排序实现的接口
2
3 // 插入排序
4 void InsertSort(int* a, int n);
5
6 // 希尔排序
7 void ShellSort(int* a, int n);
8
9 // 选择排序
```

$$\mathcal{L}_{\text{SFT}}^t = -\log \pi_{\theta}(a_{t,i}|s_t) \quad \pi_{\theta}(a_{t,i}|s_t) = \frac{\exp z_{\theta}(a_{t,i}|s_t)}{\sum_k \exp z_{\theta}(a_{t,k}|s_t)}$$

Outline

- ❑ From Text to Computer Representation
- ❑ From Unlimited Text to a Finite Vocabulary
- ❑ **BUILDING TOKENIZERS AND VOCABULARIES**
- ❑ From Token IDs to Embeddings
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ Takeaways

Tokenization: Why Tokenization Matters

❑ **Tokenization** (分词): Converting a string of text into a sequence of **smaller units** (words, subwords) called **tokens** (词元).

❑ **Function:**

- Bridge between *human-readable text* and *machine-readable numbers*.
- A suitable word segmentation strategy can generate more effective *feature representations*, which directly affects downstream tasks.
- One of the key steps in solving the **OOV (Out-of-Vocabulary)** problem.

Tokenization: Levels of Tokenization

- ❑ **Character-level:** Splits text into **individual characters** (字符).
 - **Pros (优点):** No OOV words.
 - **Cons (缺点):** Creates very long sequences.
 - **Example:** **text** → ['t', 'e', 'x', 't'].
- ❑ **Word-level:** Splits text into **words** based on spaces and punctuation.
 - **Pros:** Intuitive (直观的) and meaningful (有意义的) units.
 - **Cons:** Large vocabulary and significant OOV problems.
 - **Example:** **the cat sat** → ['the', 'cat', 'sat'].

Tokenization: Levels of Tokenization

- ❑ **Subword-level:** Breaks words into smaller, meaningful parts.
 - **Pros:** Balances vocabulary size and sequence length, nearly eliminating OOV issues.
 - **Example:** refreshing → ['re', 'fresh', 'ing']

Sentence: “Revisiting the parks after the storm was refreshing.”

- Word-level tokenization: ['Revisiting', 'the', 'parks', 'after', 'the', 'storm', 'was', 'refreshing'].
- Subword-level tokenization: ['Re', 'visit', 'ing', 'the', 'park', 's', 'after', 'the', 'storm', 'was', 're', 'fresh', 'ing'].

Word Tokenization: Space-based Tokenization

□ A very simple way to tokenize

- For languages that use space characters between words.
 - *Arabic, Greek, Latin*, etc., based writing systems.
- Segment off a token between instances of spaces.

□ Unix/Linux tools for space-based tokenization

- The “tr” command.
- Given a text file, output the word tokens and their frequencies.

Word Tokenization: Space-based Tokenization

❑ Can't just blindly remove punctuation (标点符号):

- m.p.h., Ph.D., AT&T, cap'n
- prices (\$45.55)
- dates (01/02/06)
- URLs (<http://www.stanford.edu>)
- hashtags ([#nlproc](#))
- email addresses (someone@cs.colorado.edu)

❑ A word that doesn't stand on its own

- “are” in [we're](#)
- [New York, Washington D.C.](#)

Word Tokenization: Space-based Tokenization

❑ Use **Natural Language Toolkit (NLTK)** for tokenization:

```
pip install nltk==3.4.5
```

```
1 from nltk.tokenize import word_tokenize
```

```
2 sentence = "I love NLP and LLM!"
```

```
3 words = word_tokenize(sentence)
```

```
4 print(words)
```

```
['I', 'love', 'NLP', 'and', 'LLM', '!']
```

Word Tokenization: Space-based Tokenization

❑ Use **Natural Language Toolkit (NLTK)** for tokenization:

```
pip install nltk==3.4.5
```

```
1  from nltk.tokenize import word_tokenize
2  sentence = "Prof. Quan loves NLP and LLM!"
3  words = word_tokenize(sentence)
4  print(words)
```

```
['Prof.', 'Quan', 'loves', 'NLP', 'and', 'LLM', '!']
```

Word Tokenization: Space-based Tokenization

❑ Use **Natural Language Toolkit (NLTK)** for tokenization:

```
pip install nltk==3.4.5
```

```
1 from nltk.tokenize import word_tokenize
```

```
2 sentence = "我喜欢学人工智能"
```

```
3 words = word_tokenize(sentence)
```

```
4 print(words)
```

```
['我喜欢学人工智能']
```

Tokenization in languages **without spaces**

❑ Many languages (like *Chinese, Japanese, Thai*) don't use spaces to separate words!

■ Space-based: *I like studying artificial intelligence.*

■ Without-space :

• *Chinese*: 我喜欢学习人工智能

• *Japanese*: 私は人工知能を学ぶのが好きです

• *Thai*: ฉันชอบเรียนเกี่ยวกับปัญญาประดิษฐ์

Question

How do we decide where the token boundaries should be?

Tokenization in languages without spaces

姚明进入总决赛 “Yao Ming reaches the finals”

□ 3 words?

- 姚明 进入 总决赛
- YaoMing reaches finals

□ 5 words?

- 姚 明 进入 总 决赛
- Yao Ming reaches overall finals

□ 7 words? (don't use words at all):

- 姚 明 进 入 总 决 赛
- Yao Ming enter enter overall decision game

Tokenization in languages **without spaces**

- ❑ In Chinese, we can just treat each **character** (hanzi) as a token, or we can segment the sentence into Chinese **words**.
 - **Chinese word segmentation** (中文分词) is an important task in Chinese language processing.
- ❑ Standard algorithms are models by **supervised learning** (监督学习).

<https://hanlp.hankcs.com/demos/tok.html>

LLM Tokenization

LLM Tokenization: Why we need subword?

- ❑ **Vocabulary Explosion** (词表爆炸): Word tokenization consumes significant memory and computing resources.
- ❑ **Out-of-Vocabulary, OOV** (未登录词): Word tokenization leads to unknown token **<UNK>** that causes significant information loss.
- ❑ **Morphological Relationship** (形态学关联): **run**, **running**, and **ran** are *semantically related*. Word-based models learn them as *independent tokens* and cannot utilize their shared root “**run**”.

Point

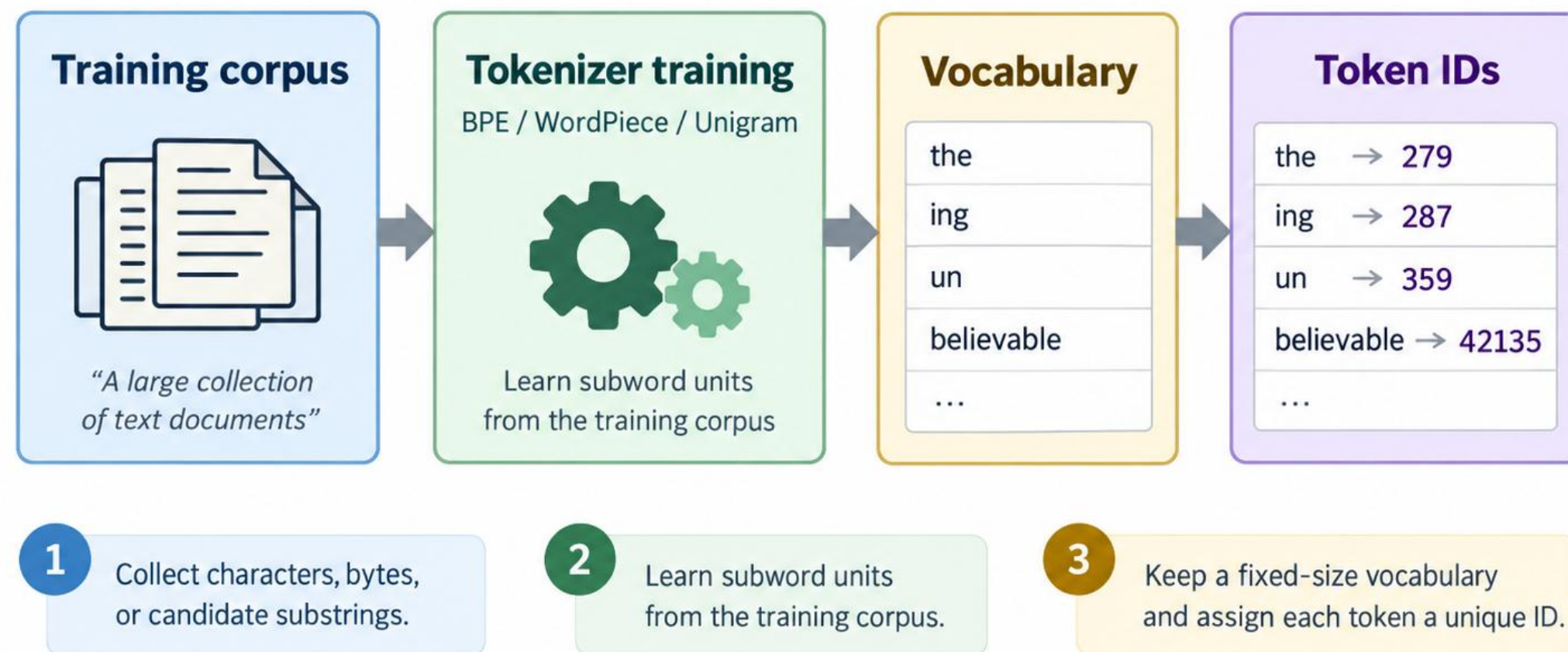
Subword algorithms achieve an excellent **balance** between **character-level tokenization** and **word-level tokenization**.

Tokenization: Subword Methods

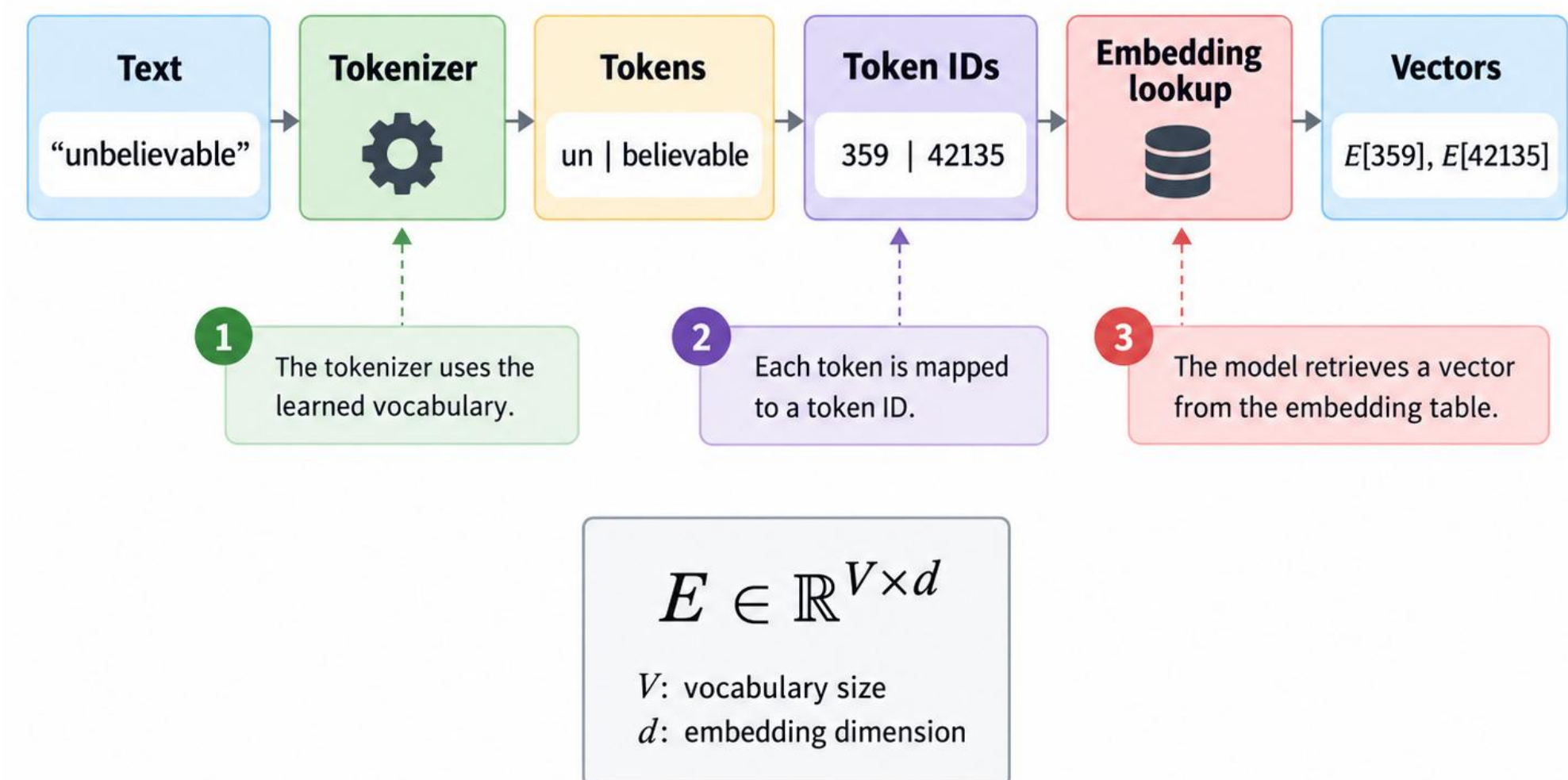
- ❑ **Byte-Pair Encoding (BPE):** Starts with characters and iteratively merges the most frequent pair of tokens.
 - Example: $t + h \rightarrow th$, then $th + er \rightarrow ther$.
- ❑ **WordPiece:** Similar to BPE, but merges pairs that maximize the likelihood of the training data, not just frequency.
- ❑ **Unigram Language Model:** Starts with a large candidate vocabulary and progressively removes less useful subwords.

From Tokenization to Vocabulary

Vocabulary Learning



Vocabulary Usage



Vocabulary Handling: OOV and Rare Words

- ❑ **The <UNK> Token Problem:** Not in the vocabulary; “Unknown”
 - Degraded performance on downstream tasks
 - Poor context comprehension
- ❑ **Subwords as the Solution:**
 - Subword tokenization effectively alleviates the <UNK> problem.



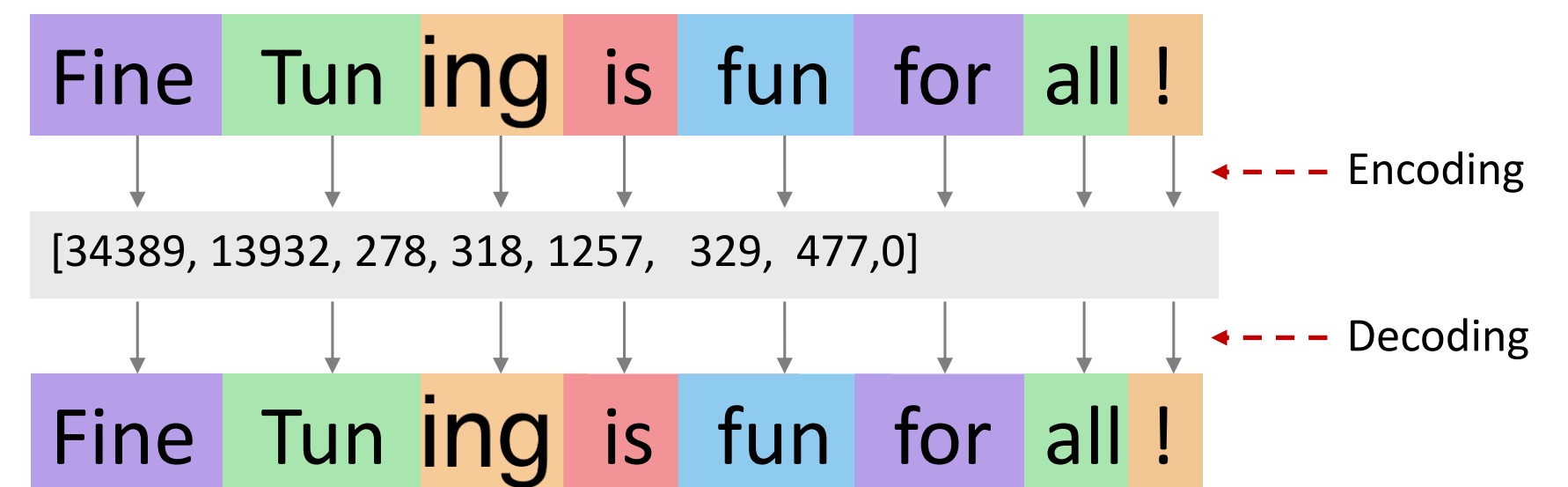
Vocabulary Handling: Special Tokens

□ Special Tokens:

- [PAD]: Unify the sequence lengths for batch processing
- [BOS]/[EOS]: The start and end of the token sequence
- [MASK]: Used for MLM pre-training tasks
- [CLS]: Used for classification tasks
- [UNK]: Used for unknown tokens

Vocabulary Handling: Encode and Decode

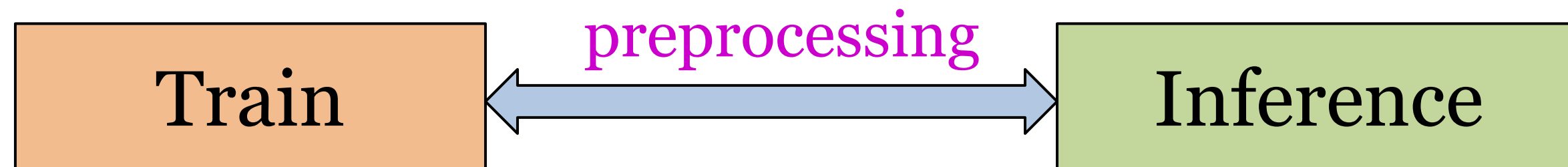
- ❑ **token** → **token id**: The vocabulary maps each **string token** to an **integer ID**. The token id is used as an index to look up the corresponding vector in the model's embedding matrix.
- ❑ **token id** → **token**: When a model generates token IDs (e.g., during text generation), these integer identifiers reference corresponding entries in a predefined vocabulary.



Vocabulary Handling: Encode and Decode

❑ Maintaining Consistency

- The entire preprocessing and tokenization pipeline (e.g., lowercasing, normalization rules, tokenizer type) must be **identical during both training and inference** to prevent performance degradation.



Outline

- ❑ From Text to Computer Representation
- ❑ From Unlimited Text to a Finite Vocabulary
- ❑ Building Tokenizers and Vocabularies
- ❑ **FROM TOKEN IDS TO EMBEDDINGS**
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ Takeaways

Word Embeddings: Concept

□ Core Idea: Learning vectors from text for representing words

■ **Input:**

- V : a pre-defined vocabulary.
- d : dimension of word vectors.
- A large text corpora

■ **Output:** $f: V \rightarrow \mathbb{R}^d$

$$v_{\text{cat}} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ 0.276 \end{pmatrix} \quad v_{\text{dog}} = \begin{pmatrix} -0.124 \\ 0.430 \\ -0.200 \\ 0.329 \end{pmatrix}$$

$$v_{\text{the}} = \begin{pmatrix} 0.234 \\ 0.266 \\ 0.239 \\ -0.199 \end{pmatrix} \quad v_{\text{language}} = \begin{pmatrix} 0.290 \\ -0.441 \\ 0.762 \\ 0.982 \end{pmatrix}$$

Word Embeddings: **Word2vec**

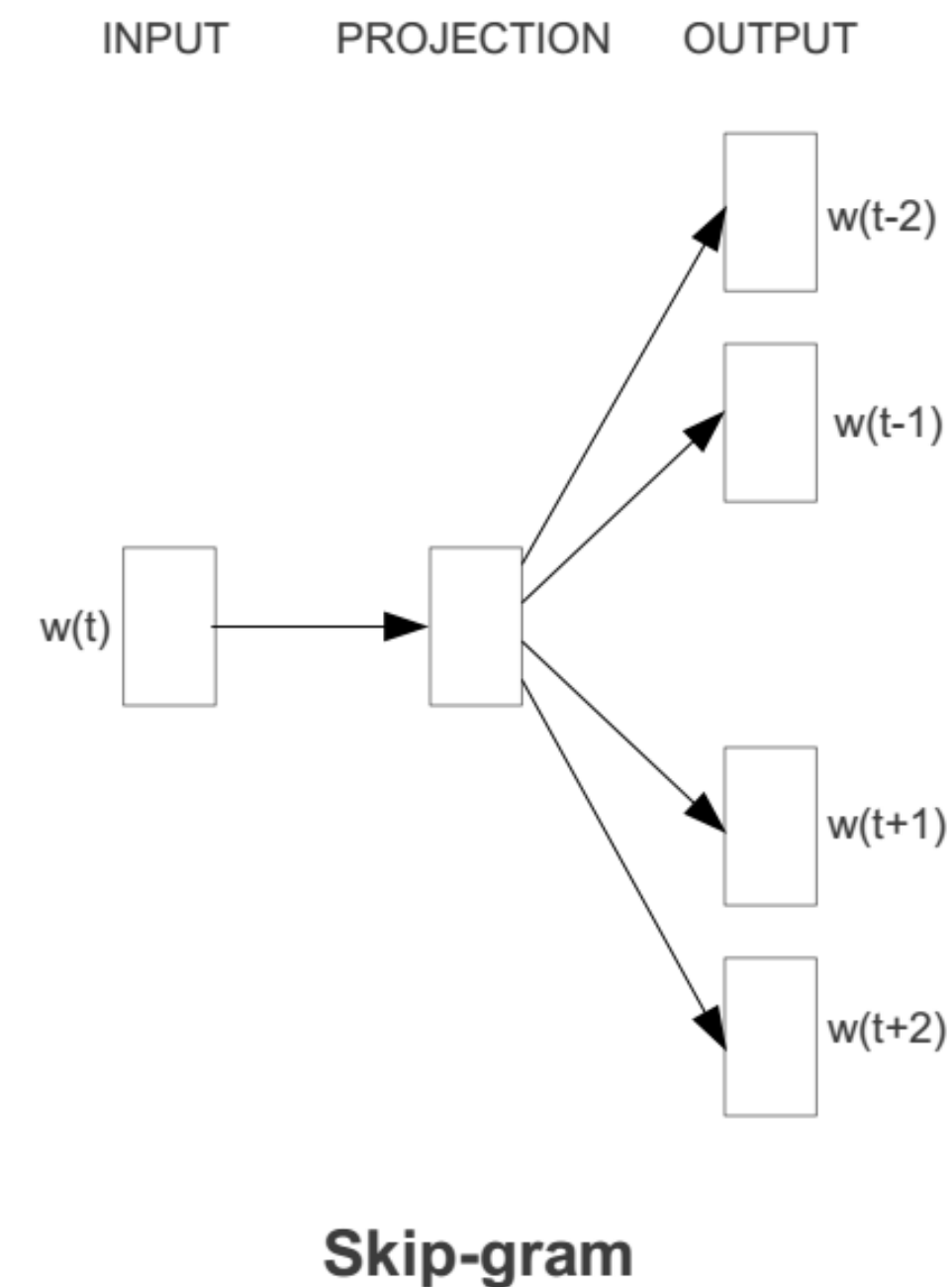
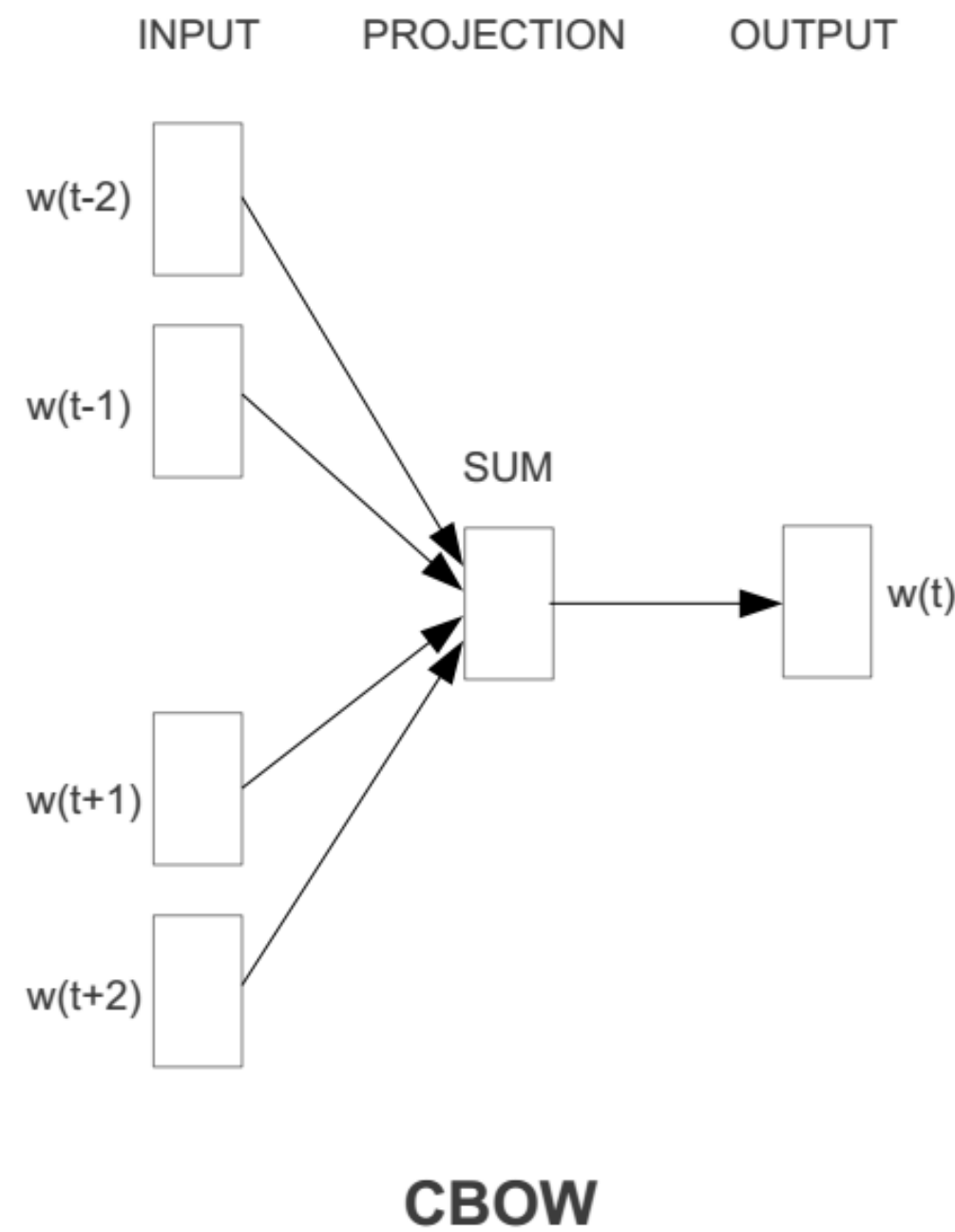
□ Core Idea: Learn word vectors, **predict** rather than count.

- We have a large corpus of text: a long list of words.
- Every word in a fixed vocabulary is represented by a **dense vector**.
- Go through each position t in the text, which has a center word c and context (“outside”) words o .
- Use the **similarity of the word vectors** for c and o to **calculate the probability** of o given c (or vice versa).
- **Keep adjusting the word vectors** to maximize this probability.

Word Embeddings: **Word2vec**

□ Two basic neural network models:

- **CBOW (Continuous Bag of Words)**: Predict target word from context.
- **Skip-gram**: Predict context words from target



Word Embeddings: **Word2vec**

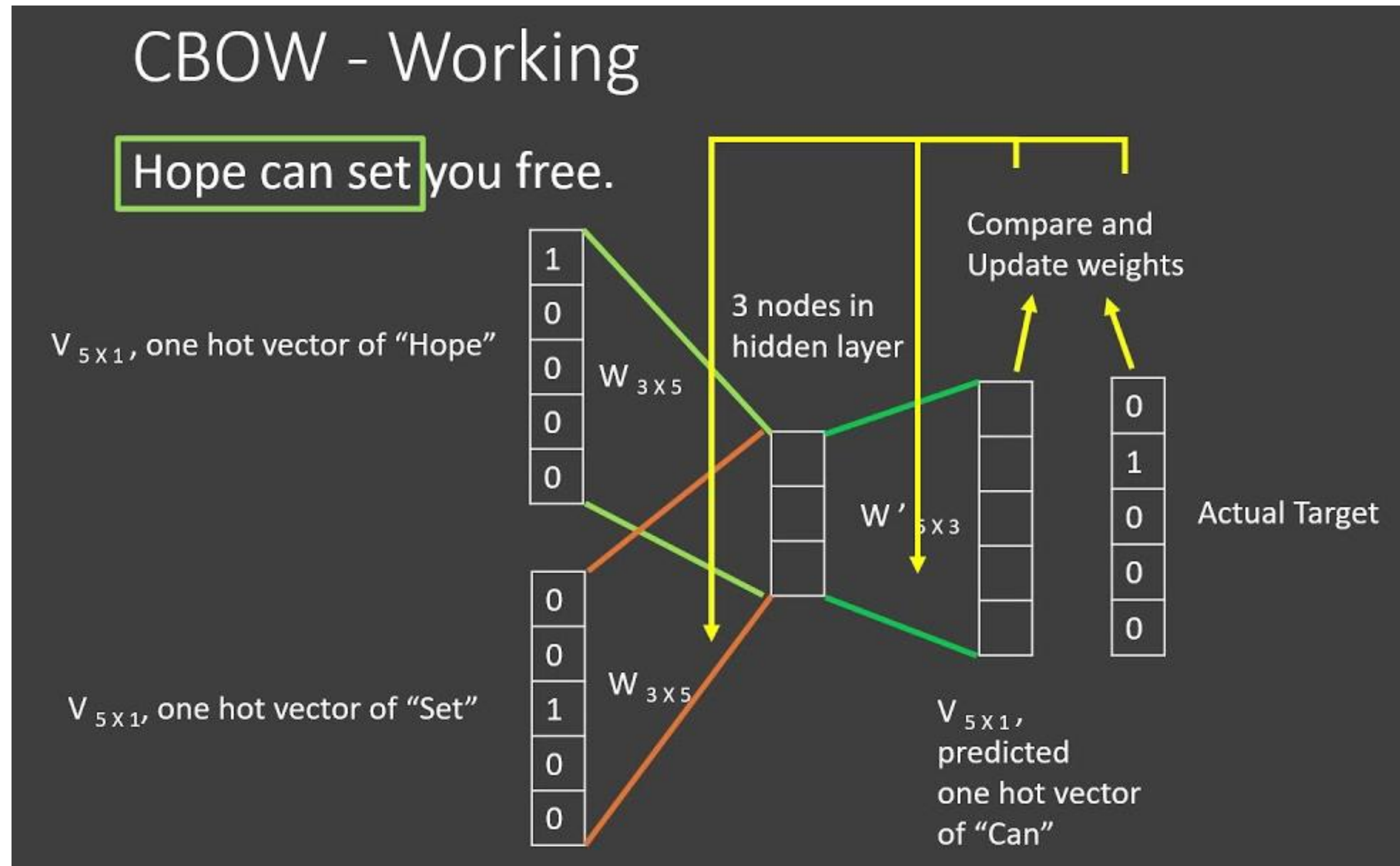
❑ **CBOW**: Predict target word from context.

- **Input Layer**: Represents the **context words** encoded as one-hot vectors
- **Hidden Layer**: Processes the input and performs nonlinear transformations to capture the semantic relationships between words.
- **Output Layer**: **Produces a probability distribution** over the vocabulary.

Definition

A **one-hot vector** represents a token as a vector with one element equal to **1** and all others equal to **0**.

Word Embeddings: Word2vec

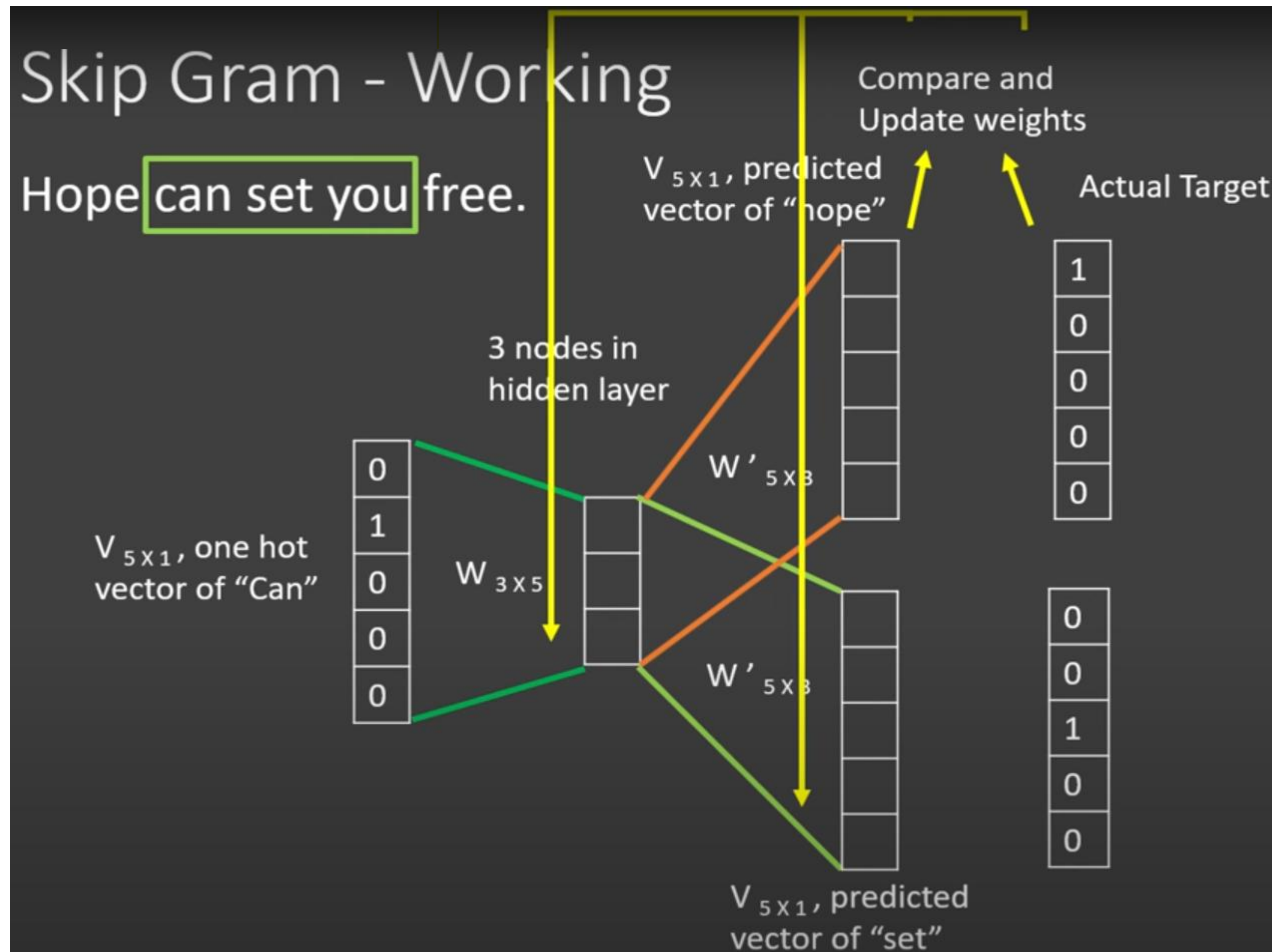


Word Embeddings: **Word2vec**

❑ **Skip-gram**: Predict context words from target

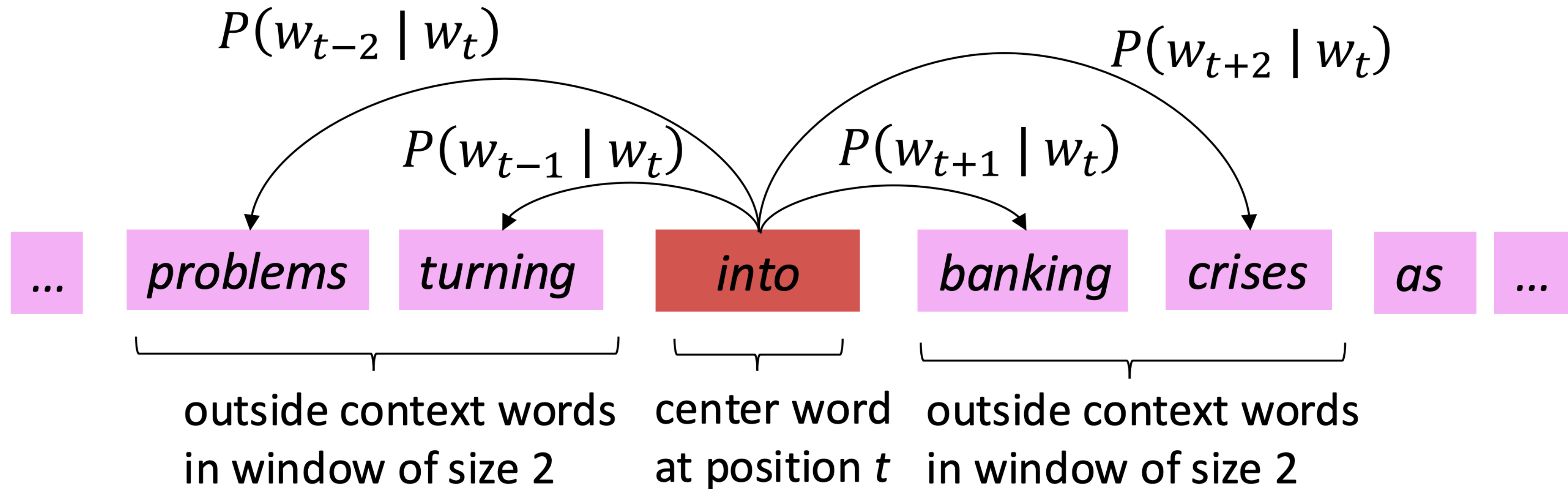
- **Input Layer**: Takes a single word encoded as a one-hot vector.
- **Hidden Layer**: Transforms the input word into a distributed representation in the hidden layer.
- **Output Layer**: Predicts the context words based on the representation learned in the hidden layer.

Word Embeddings: Word2vec



Word Embeddings: Word2vec: Skip-gram

□ Example windows and process for computing $P(w_{t+j} | w_t)$



Word Embeddings: Word2vec: Skip-gram

□ Skip-gram with **negative sampling** (SGNS)

- Simplify to a binary classification task:
 - **Positive Samples**: Context words that **actually appear** around the target word.
 - **Negative Samples**: **Randomly** selected words **NOT appear** in the context.
- Goal: **Maximize** probability of positive samples, **minimize** probability of negative samples.

Word Embeddings: Word2vec: Skip-gram

- ❑ Instead of **counting** how often each word w occurs near “*bank*”,
 - Train a binary classifier: Is w likely to show up near “*bank*”?
- ❑ We don't actually care about this task,
 - But we'll take the learned classifier weights as the word embeddings.
- ❑ Big idea: self-supervision (自监督):
 - A word c that occurs near *bank* in the corpus as the gold “*correct answer*” for supervised learning.
 - No need for human labels.

Word Embeddings: Word2vec: Skip-gram

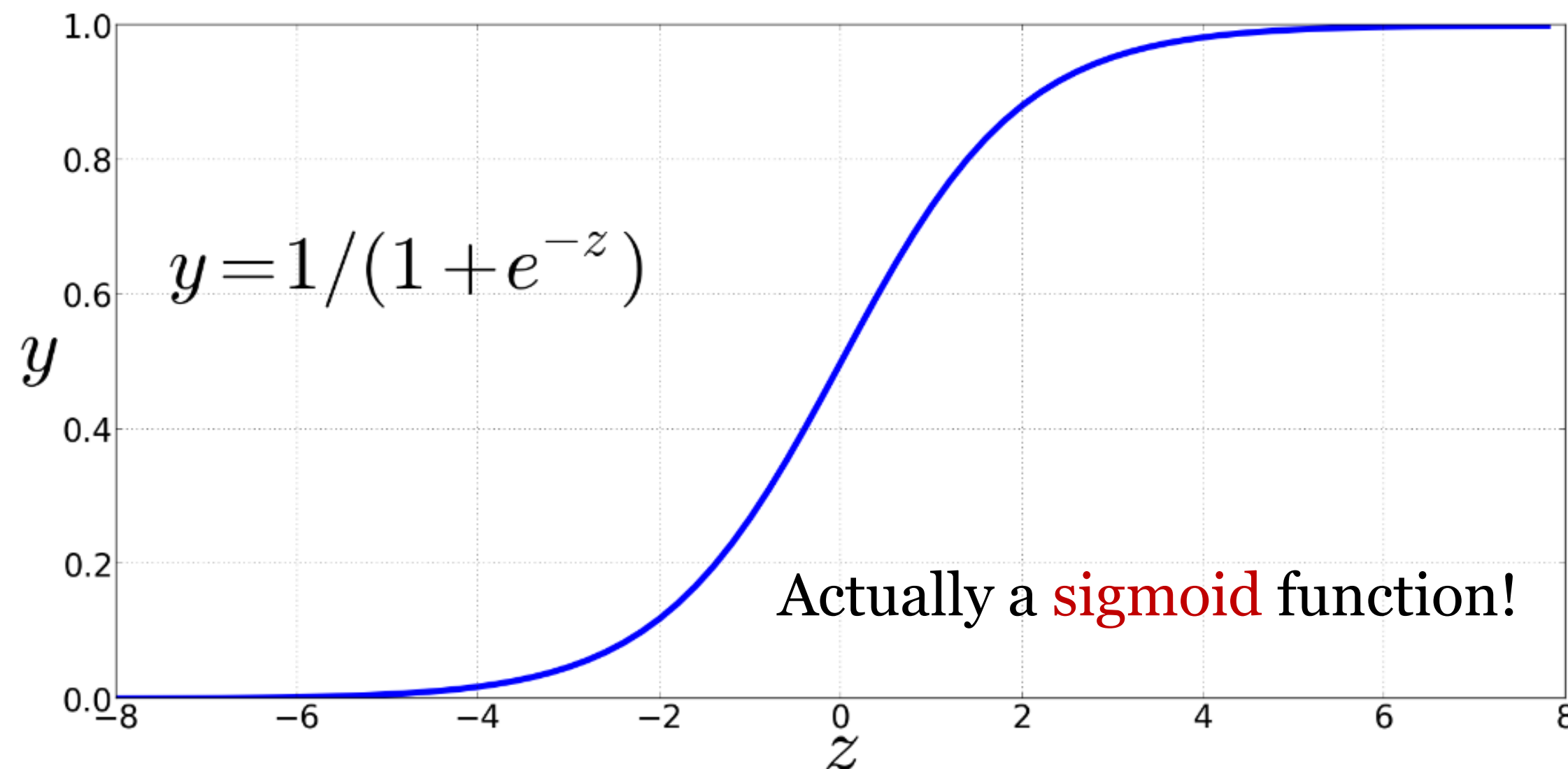
- Predict if candidate word c is a “neighbor”.
- Treat the target word t and a context word c as positive examples.
- Randomly sample other words in the vocabulary to get negative examples.
- Train a logistic regression classifier to distinguish those two cases.
- Use the learned weights as the embeddings.

Q: Why don't we simply predict context words given target word?

Because predicting all context words via softmax is costly. Thus, SGNS uses logistic regression to efficiently learn embeddings by distinguishing real (target, context) word pairs from randomly sampled fake pairs.

Word Embeddings: Logistic Regression

- ❑ **Logistic regression** is a classification algorithm that models the probability of a binary outcome (e.g., true/false) using the logistic (sigmoid) function to map inputs to values between 0 and 1.



Word Embeddings: Word2vec: Skip-gram

□ For example: assume a ± 2 word window, given training sentence:

...lemon, a tablespoon of apricot jam, a pinch...

c1 c2 **[target]** c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Word Embeddings: Word2vec: Skip-gram

□ For example: assume a +/- 2 word window, given training sentence:

...lemon, a tablespoon of apricot jam, a pinch...

c1 c2 [target] c3 c4

□ **Goal:** train a classifier given candidate (word, context) pairs

(apricot, jam)

(apricot, like) ...

□ And assigns each pair a probability:

$$P(+|w, c)$$

$$P(-|w, c) = 1 - P(+|w, c)$$

Word Embeddings: **Word2vec**

□ Similarity is computed from **dot product**.

- Remember: two vectors are similar if they have a high dot product
 - Cosine is just a normalized dot product
- So:
 - $\text{Similarity}(w, c) \propto w \cdot c$
- We'll need to **normalize** to get a probability
 - (cosine isn't a probability either)

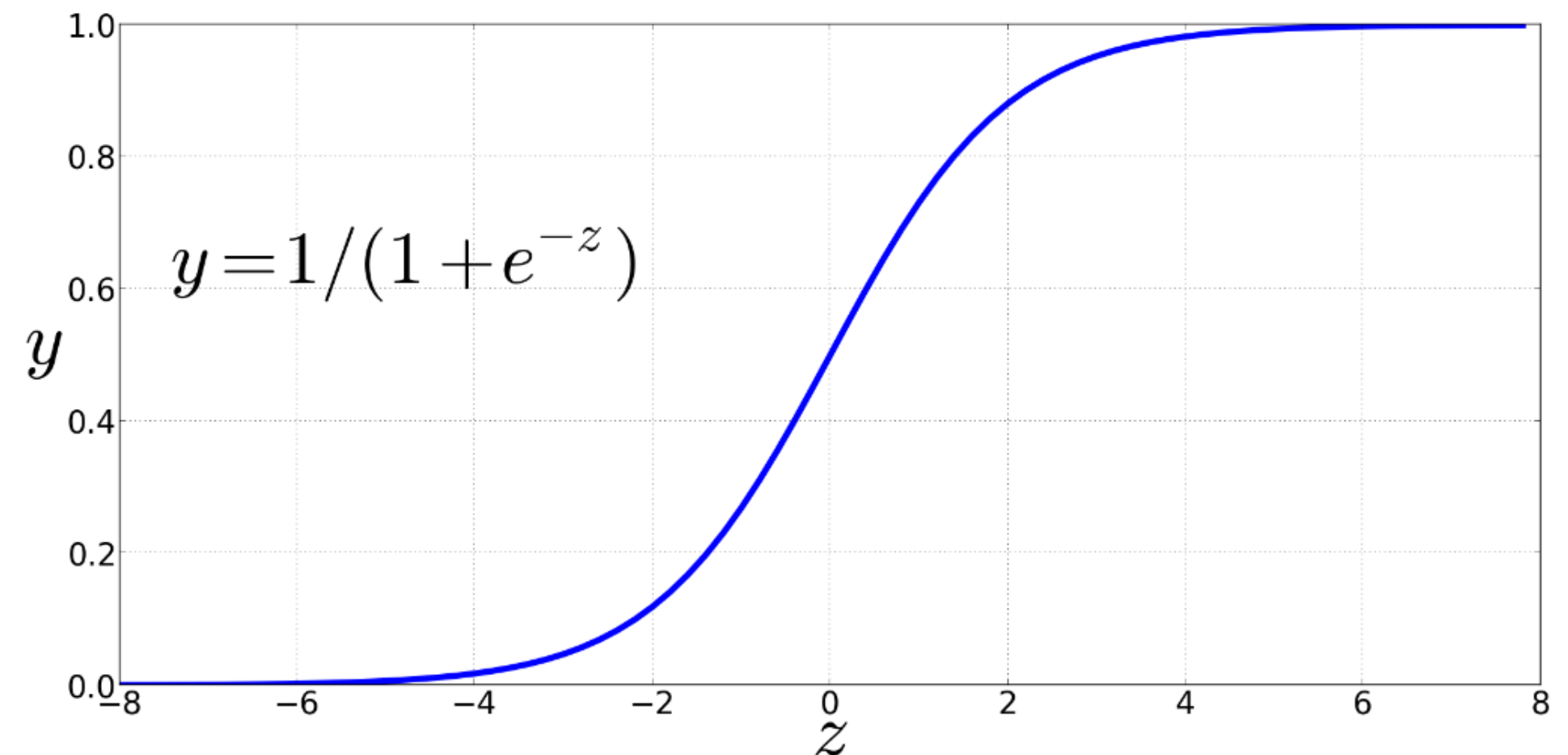
Word Embeddings: **Word2vec**

□ Turn dot products into probabilities.

- $\text{Sim}(w, c) \approx w \cdot c$
- To turn this into a probability
- We'll use the **sigmoid** from logistic regression:

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

$$\begin{aligned} P(-|w, c) &= 1 - P(+|w, c) \\ &= \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)} \end{aligned}$$



Word Embeddings: **Word2vec**

□ How skip-gram classifier computes $P(+|w, c)$

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

- This is for one context word, but we have lots of context words.
- We'll **assume independence** and just multiply them:

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

Word Embeddings: **Word2vec**

□ Loss function

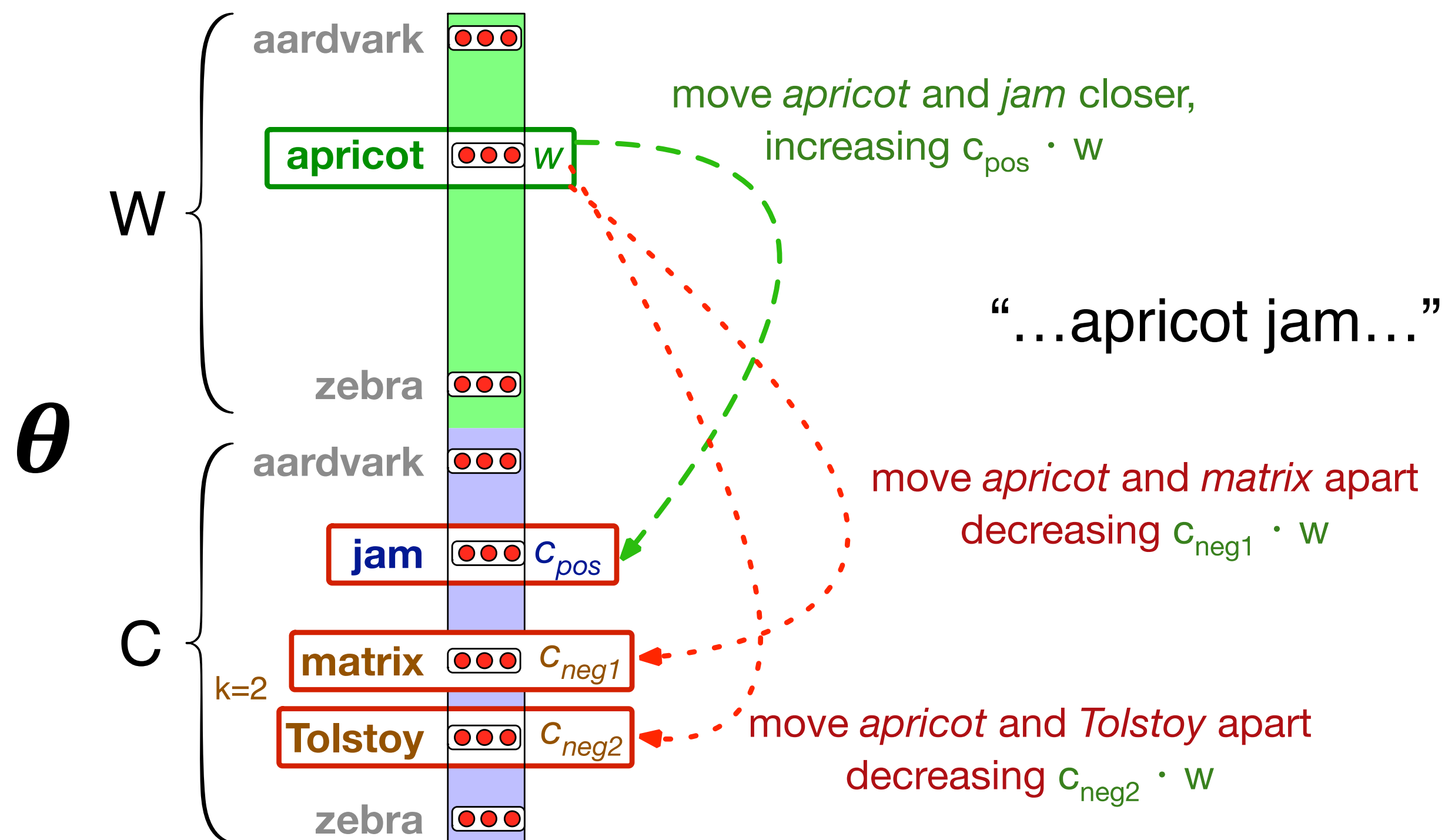
- **Maximize** the similarity of the target with the actual context words
- **Minimize** the similarity of the target with the *k* negative sampled non-neighbor words.

$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

Word Embeddings: Word2vec

□ How to learn the classifier?

■ **Stochastic gradient descent!** (随机梯度下降)



Word Embeddings: **Word2vec**

□ At each step of gradient descent

- **Direction**: move in the reverse direction from the gradient of the loss function
- **Magnitude (大小)**: we move the value of this gradient $\frac{d}{dw} L(f(x; w), y)$ weighted by a **learning rate η**
- **Higher learning rate (学习率)** means move w faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

Word Embeddings: **Word2vec**

□ The derivatives of the loss function

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Word Embeddings: **Word2vec**

□ Start with **randomly initialized** C and W matrices, then update:

$$c_{pos}^{t+1} = c_{pos}^t - \eta [\sigma(c_{pos}^t \cdot w^t) - 1] w^t$$

$$c_{neg}^{t+1} = c_{neg}^t - \eta [\sigma(c_{neg}^t \cdot w^t)] w^t$$

$$w^{t+1} = w^t - \eta \left[[\sigma(c_{pos} \cdot w^t) - 1] c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w^t)] c_{neg_i} \right]$$

Word Embeddings: Word2vec

□ SGNS learns two sets of embeddings

- Target embedding matrix W
- Context embedding matrix C

□ It's common to just add them together, representing word i as the vector $w_i + c_i$

Question: Why don't we simply use one shared embedding matrix?

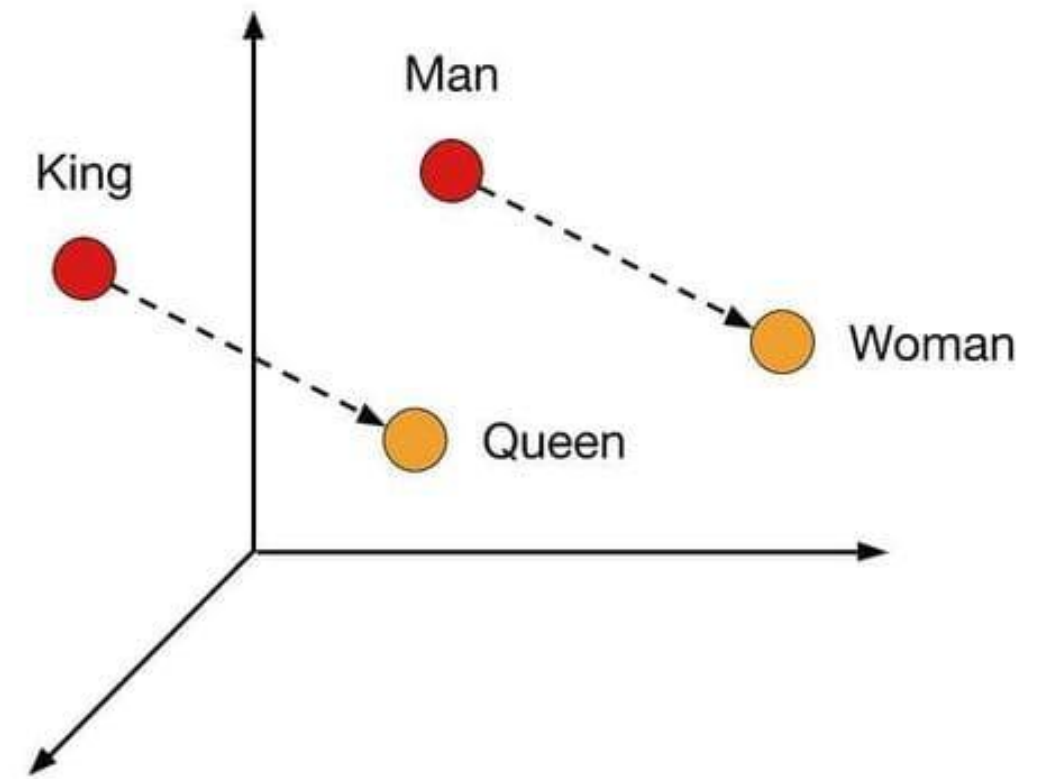
Target and context words play **different roles** in prediction. Keeping two matrices allows the model to learn **asymmetric, richer relationships** between words and yields better performance. A **shared matrix would limit expressiveness** and hurt embedding quality.

Word Embeddings: Word2vec

- Word2Vec embeds words into a continuous vector space where semantic relationships become linear.
- This means relationships between words can be captured through vector arithmetic (算术运算).

$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}}$ is close to $\vec{\text{queen}}$

$\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}}$ is close to $\vec{\text{Rome}}$



$$\text{King} - \text{Man} + \text{Woman} = \text{Queen}$$

Outline

- ❑ From Text to Computer Representation
- ❑ From Unlimited Text to a Finite Vocabulary
- ❑ Building Tokenizers and Vocabularies
- ❑ From Token IDs to Embeddings
- ❑ **CONTEXTUAL REPRESENTATIONS**
- ❑ Practice & Discussion
- ❑ Takeaways

Why One Fixed Embedding Is **Not Enough**

- ❑ A token retrieves the same input embedding, even when its meaning changes with context.



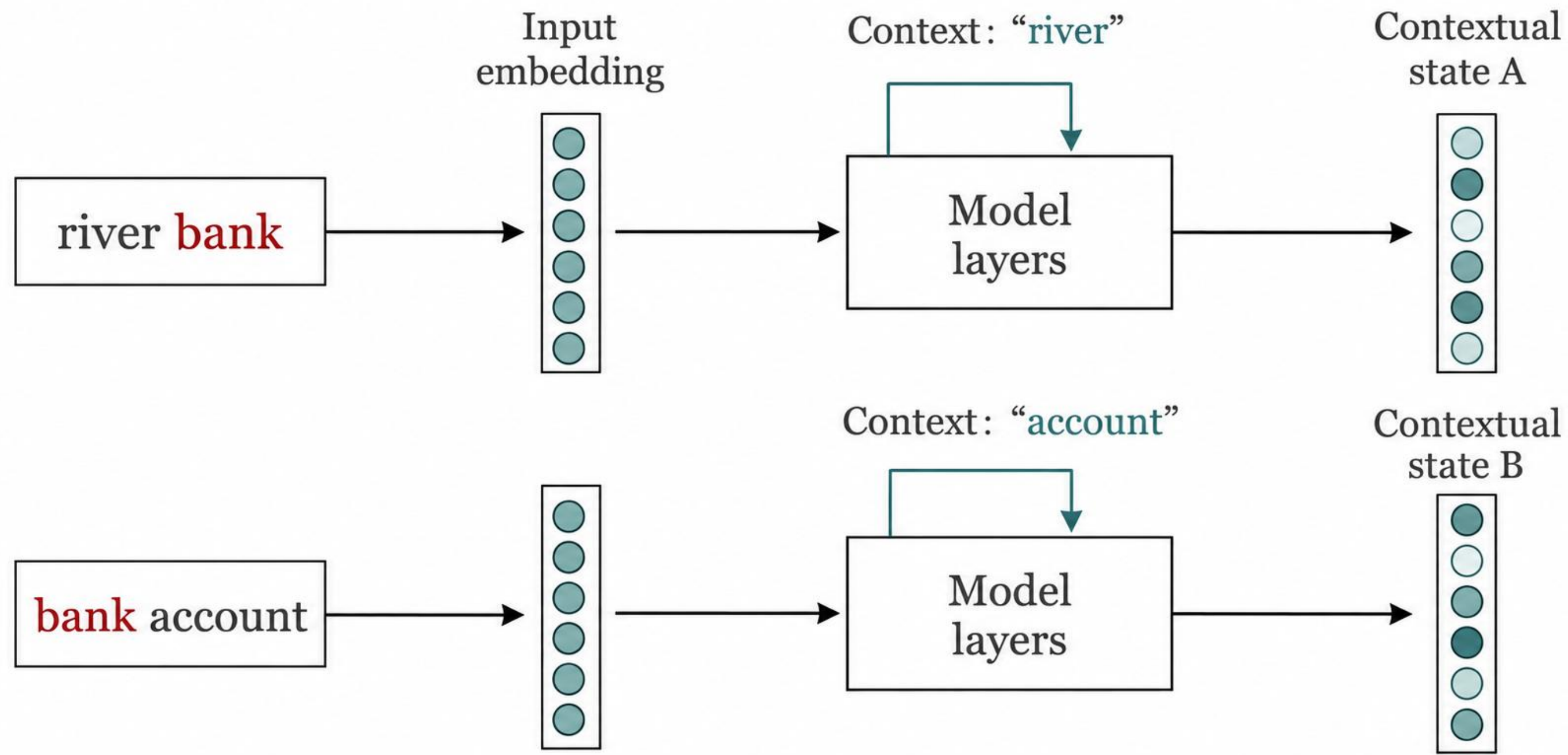
river bank



bank account

Input Embedding vs. Contextual State

- ❑ Embedding lookup provides a starting vector. Model layers combine it with sequence information.



Same token ID and input embedding. Different context and hidden state.

Context Changes the Representation

- ❑ Surrounding tokens provide evidence about the current meaning and role.

Sentence	Contextual interpretation of “bank”
The fisherman sat on the <i>bank</i> .	Land beside a river
She deposited money in the <i>bank</i> .	A financial institution

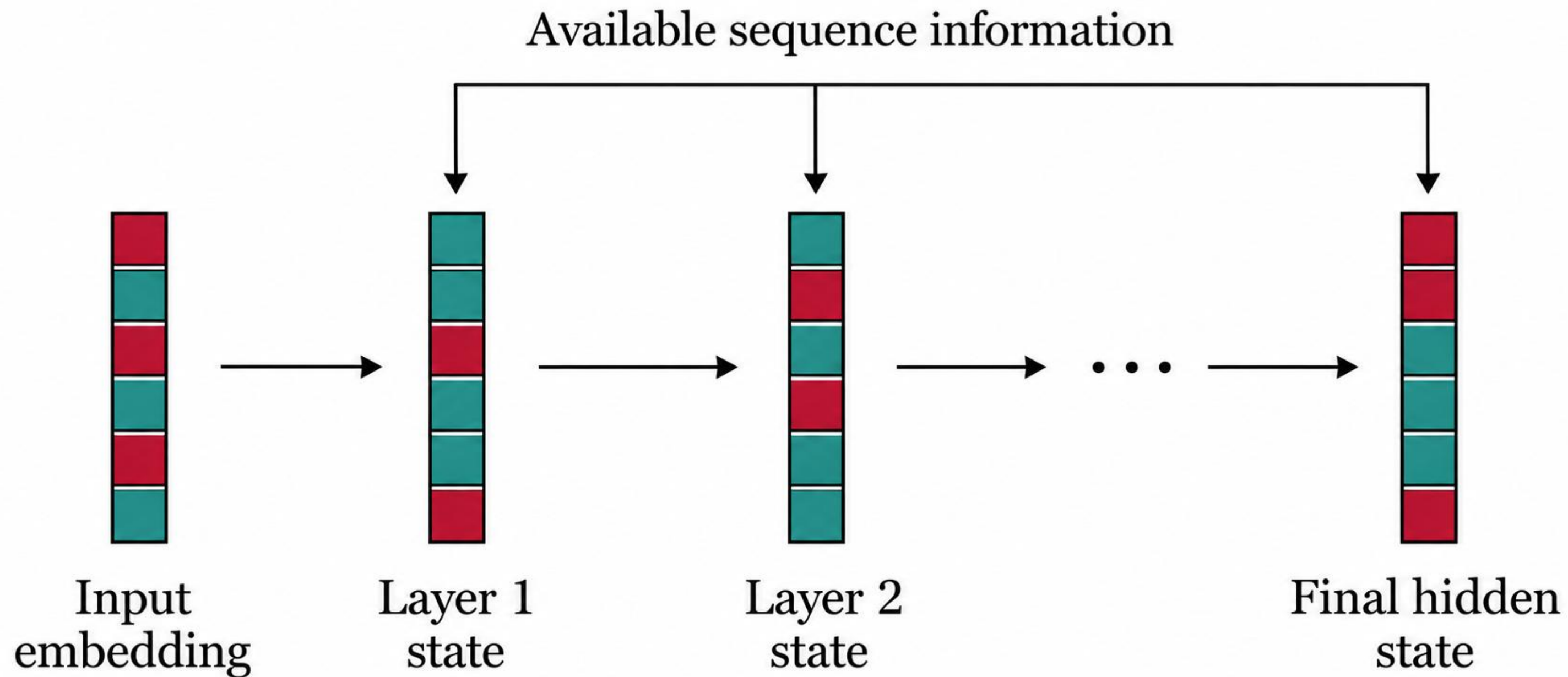
$$\mathbf{h}_i = \mathbf{f}(\mathbf{x}_i, \text{context})$$

Key idea

The resulting representation depends on both the token and its context.

Contextualization Happens Layer by Layer

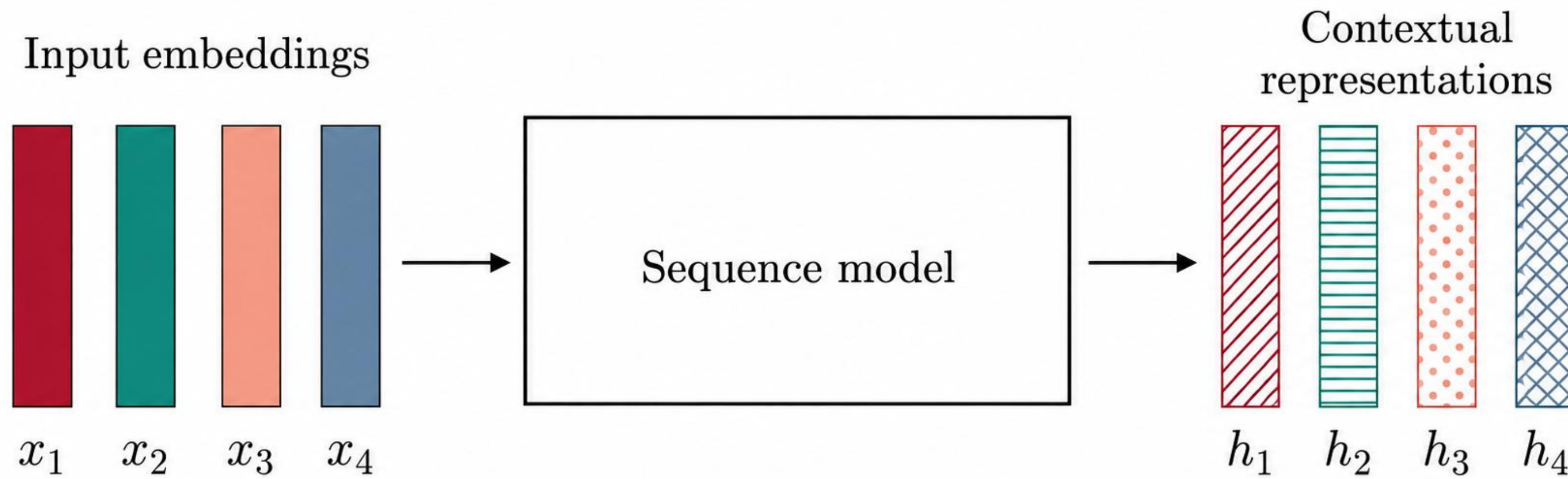
- ❑ Each layer updates the representation using the sequence information available to it.



Each layer updates the representation built by earlier layers.

Representations Need Sequence Modeling

- Contextual representations require a mechanism that combines information across positions.



How does information move across positions?

Outline

- ❑ From Text to Computer Representation
- ❑ From Unlimited Text to a Finite Vocabulary
- ❑ Building Tokenizers and Vocabularies
- ❑ From Token IDs to Embeddings
- ❑ Contextual Representations
- ❑ **PRACTICE & DISCUSSION**
- ❑ Takeaways

Mini Lab: Text to Representations

- ❑ Three experiments connect model inputs with learned representations.

Part A	Part B	Part C
Tokenization	Embedding	Contextual representation
Model-specific units	Learned vector geometry	Context-dependent states

1. Predict	2. Run the tool	3. Record evidence	4. Explain
------------	-----------------	--------------------	------------

Key idea

Measured evidence comes before interpretation.

Tools and Resources

- ❑ Use one shared setup so that group results remain comparable.

Resource	Classroom use	Access
Hugging Face Tokenizer Playground	Inspect tokens and token IDs	huggingface.co/spaces/Xenova/the-tokenizer-playground
Gensim KeyedVectors	Nearest neighbors and cosine similarity	radimrehurek.com/gensim/models/keyedvectors.html
TensorFlow Embedding Projector	Explore vector neighborhoods visually	projector.tensorflow.org
Lecture2_Representation_Lab_Colab.ipynb	Consistent course workflow	Provided by the instructor

Key idea

All groups use the same tokenizer and pretrained embedding model.

Activity 1: Tokenization Prediction and Test

Predict first. Then record the tokenizer output exactly.

Text	Prediction	Actual tokens	Token IDs	# tokens
playing				
unbelievable				
ChatGPT				
深圳河套学院				
Let's eat, grandma.				
Let's eat grandma.				

Activity 1: Tokenization Observations

❑ Explain what changed and cite an example from your tokenizer results.

Observation	Example from your results	Interpretation
A word split into reusable subwords		
Punctuation changed the token sequence		
Different scripts used different token counts		
Your prediction differed from the tokenizer		

Key idea

What representation decisions occur before the neural model sees the text?

Activity 2: A Real Embedding Space

Retrieve Top-5 neighbors, then measure the selected word pairs.

Target	Top-5 neighbors	Relationship type(s)	Evidence or surprise
dog			
happy			
running			
Paris			
doctor			

Cosine pairs	dog and cat, dog and car, happy and sad, happy and joyful
Continue	run and running, doctor and hospital, Paris and France

How Can an Embedding Express Meaning?

- ❑ Use measured neighbors and cosine similarities as evidence.

Discussion question	Evidence from your experiment
If an embedding is a list of numbers, where is meaning?	
Does one dimension have one human-readable meaning?	
Why can dog and cat be close?	
Why can happy and sad also be relatively close?	
What does vector position show that one value cannot?	

Key idea

Write a 2–3 sentence explanation and cite at least two measured examples.

Activity 3: Same Token, Different Context

❑ Compare bank in three contexts: money deposit, river location, and loan approval.

Evidence	Observed result	Interpretation
Token ID of bank		
Input embedding similarity		
Contextual representation similarity		
Most similar contextual pair		

Key idea

What additional information appears only after contextualization?

Final Group Submission

Submit one evidence-based slide with three experimental rows.

Experiment	Observation	Measured evidence	What we learned
Tokenization			
Embedding and semantics			
Contextual representation			

Minimum evidence	Required amount
Concrete tokenization outputs	2, including tokens and IDs
Nearest-neighbor results	2 target words
Cosine-similarity measurements	3 word pairs
Embedding meaning explanation	1 explicit explanation
Contextual comparison	1 comparison

Outline

- ❑ From Text to Computer Representation
- ❑ From Unlimited Text to a Finite Vocabulary
- ❑ Building Tokenizers and Vocabularies
- ❑ From Token IDs to Embeddings
- ❑ Contextual Representations
- ❑ Practice & Discussion
- ❑ **TAKEAWAYS**

The Complete Representation Pipeline

Running example: “*The bank is open.*”

Stage	Example or role
Character encoding and bytes	UTF-8 stores the sentence as a byte sequence.
Optional preprocessing	May normalize surface variation before tokenization.
Tokenization	The / bank / is / open / . (toy segmentation)
Vocabulary IDs	17, 205, 9, 88, 3 (illustrative IDs)
Embedding lookup	E[17], E[205], E[9], E[88], E[3]
Sequence layers	Context-dependent states h_1, h_2, h_3, h_4, h_5

Seven Ideas to Take Away

- ❑ Encoding and tokenization solve different problems.
- ❑ Tokens are model-specific units, not necessarily words or characters.
- ❑ A finite vocabulary trades sequence length, coverage and parameter cost.
- ❑ Token IDs are arbitrary labels and carry no semantics.
- ❑ Input embeddings are learned starting representations.
- ❑ Static embeddings cannot fully express meaning in the current context.
- ❑ Contextual representations change as the model combines sequence infor.

One Question to Carry Forward

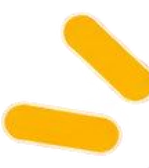
❑ Token representations must change with context.

How can a model combine information across token positions?

How can it preserve the history that matters?

Key idea

Next lecture: Neural Language Models and Sequence Modeling

 **Learn** something **useful.**
 ***Enjoy*** the process.